CS 6008 — Computer Networks

Assignment 2: DNS Security

Building a DNS system, attacking it, and adding defences one layer at a time

Akshit Kumar	23b0945
Pushpendra Uikey	23b1023

1. Introduction

DNS is the service that turns a name like **example.com** into an IP address. By default, DNS messages travel in plain text and are not protected in any way. Anyone who sits on the network path between a user and the DNS servers can read the question that is being asked and send back a fake answer.

In this assignment we built a small but complete DNS system inside Docker containers. We first made it work normally, then we attacked it from a position on the path. After each attack we added one new defence and ran the same attack again. This let us see clearly what each defence stops and what it still leaves open. Nothing in the lab touches the real Internet, so the attacks are safe to run.

2. Our Setup

We used Docker to run many small machines (containers) across three separate networks. Two routers join the networks, and everything between networks must pass through a router. An attacker machine sits on the path and tries to cheat. The main machines are:

Machine	Network	Role
client	client	the user; asks DNS questions and opens web pages
webserver	client	the real example.com website
rogue-web	client	the attacker's fake website
resolver	resolver	answers the client's DNS questions (does the real work)
root-ns	authoritative	the top ('.') name server
tld-ns	authoritative	the '.com' name server
domain-ns	authoritative	the 'example.com' name server
attacker	on the path	reads and changes messages; tries to fool the client

The three networks are kept apart on purpose, so that every cross-network message passes through a router and the on-path attacks are realistic.

3. The Seven Phases

Phase 1 — Building the DNS system

Goal. Set up the full three-level DNS system (root → com → example.com) and show that a normal lookup works and that the networks are properly separated.

What we did. We placed each server at its address, pointed the resolver at our own root, and had the client look up example.com and open its page.

```
$ dig example.com                  (asked from the client)
;; status: NOERROR
;; ANSWER SECTION:
example.com.   3600  IN  A  172.16.1.20
;; SERVER: 172.16.2.10#53   (our resolver)

$ curl http://example.com
<h1>Welcome to example.com</h1>        (the real page)

$ dig @172.16.3.10 example.com         (straight to the root server)
;; UDP setup ... failed: network unreachable
;; no servers could be reached         (the networks are isolated)

# one full lookup captured on the core router:
resolver -> root-ns : NS? .         ->  root-ns.
resolver -> root-ns : NS? com.      ->  go ask the .com server
resolver -> tld-ns  : A? example.com ->  go ask the example.com server
resolver -> domain-ns: A? example.com ->  A 172.16.1.20   (the answer)
```

Our observation. Every part of a lookup is visible in plain text on the path: the name that is asked and the answer that is returned. The client cannot reach the name servers directly, so only a machine on the path (or a router) can see or change a lookup. This visibility is exactly the weakness the next phase uses.

Phase 2 — On-path fake reply attack

Goal. Put an attacker on the path and make it send the client to a fake website, while all other names keep working.

What we did. We added the attacker and a small program that watches the DNS replies and, for example.com only, replaces the address with the fake website (172.16.1.30).

```
# attacker relaying normally (no change):
example.com     -> 172.16.1.20   (real)
www.example.com -> 172.16.1.20

# attacker forging the answer:
example.com     -> 172.16.1.30   (fake)
www.example.com -> 172.16.1.20   (left untouched)

$ curl http://example.com
<h1>** example.com has been HIJACKED **</h1>

# the attacker program's own log:
[interceptor] forging: example.com A -> 172.16.1.30
[FORGED] 159 bytes -> resolver
```

Our observation. The attack works because plain DNS has no way to prove that an answer is genuine. Sitting on the path is a big advantage: the attacker can read the real reply, copy its id and port, and send a matching fake one. An attacker who could not see the traffic would have to guess those values and would usually fail. Because the program only changes example.com, every other name still works.

Phase 3 — Adding DNSSEC (digital signatures)

Goal. Sign the DNS data, make the resolver check the signatures, and run the very same attack again.

What we did. We signed each zone and linked them into one chain of trust that starts from a root key we trust. We used the ECDSA P-256 algorithm because its signatures are small and fit in a single message. No private key is kept in our project files.

```
# before attacking (no attacker on the path):
$ dig +dnssec example.com
;; flags: qr rd ra ad          <-- 'ad' = the answer is authenticated
example.com.  IN  A      172.16.1.20
example.com.  IN  RRSIG  A 13 ...   <-- a digital signature

$ delv example.com
; fully validated              <-- the whole chain checks out

# the same forgery as Phase 2, now:
example.com     -> SERVFAIL    <-- forged answer is rejected
www.example.com -> 172.16.1.20
resolver log: validating example.com/A: no valid signature found
              RRSIG failed to verify
$ curl http://example.com  ->  fetch FAILED (no fake page)
```

Our observation. DNSSEC adds a digital signature to every answer, and the resolver checks it against the chain of trust. The forged answer has no valid signature, so the resolver rejects it (SERVFAIL) and the client cannot be sent to the fake page. One important point: DNSSEC proves the answer is genuine, but it does not hide it. On the wire the attacker can still read every question and answer, exactly as in Phase 1. So DNSSEC protects against *changing* data, not against *reading* it.

Phase 4 — Removing the signatures (downgrade)

Goal. Try to make the signed zone look unsigned, so the plain fake answer is accepted again. We test two set-ups.

What we did. We taught the attacker to strip the DNSSEC records from the reply. In Run A the set-up is correct. In Run B we recreate a common operator mistake: the parent (com) never publishes the small linking record (the DS) for example.com.

```
==== RUN A : set up correctly (com has example.com's DS) ====
[STRIPPED] 206 bytes -> resolver
example.com -> SERVFAIL           (attack FAILS)
resolver log: insecurity proof failed resolving example.com/DNSKEY
$ curl http://example.com -> FAILED (no hijack)

==== RUN B : operator mistake (com has NO DS for example.com) ====
# with no attacker, example.com is now seen as unsigned:
$ dig +dnssec example.com   ->  flags: qr rd ra   (no 'ad')

# the SAME strip attack now works:
[STRIPPED] 206 bytes -> resolver
example.com -> 172.16.1.30
$ curl http://example.com -> ** example.com has been HIJACKED **
```

Our observation. In Run A the attacker cannot make the zone look unsigned, because the parent publishes a signed DS record proving that example.com must be signed, and the attacker cannot forge that signed proof. The resolver tries to prove the zone is unsigned, fails ('insecurity proof failed'), and returns SERVFAIL. In Run B the operator signed the child but never gave the DS to the parent, so there is a signed proof that example.com is unsigned; the resolver believes it and accepts the fake answer. The protection comes from the parent's signed DS; forgetting to publish it throws the protection away.

Phase 5 — Zone walking (finding hidden names)

Goal. A signed zone must prove when a name does *not* exist. Show that this proof can leak the list of real names, then fix it.

What we did. We added five secret internal names (dev, vpn, mail, db, backup) and wrote a small program that, starting from only 'example.com', follows the 'does-not-exist' proofs from one real name to the next.

```
# a name that does not exist still returns a signed proof that
# names the real neighbours on each side of the gap:
$ dig +dnssec nonexistent.example.com   ->  status: NXDOMAIN
example.com.       IN NSEC backup.example.com. ...
mail.example.com.  IN NSEC ns.example.com. ...

# the walker, starting from only 'example.com':
example.com      -> next = backup.example.com
backup.example.com -> next = db.example.com
db -> dev -> mail -> ns -> vpn -> www -> (back to example.com)
8 queries, recovered 7 names: backup, db, dev, mail, ns, vpn, www

# after the fix (re-signed with NSEC3, which uses hashes):
$ dig +dnssec nonexistent.example.com
BFNO28NKKN9K7B20...  IN NSEC3 ...   <-- owner is a hash, not a name
the SAME walk now: no NSEC in reply -> recovered 0 names
```

Our observation. A signed zone has to prove that a name does not exist, and that proof is signed in advance. With the default method (NSEC) the proof names the two existing names on either side of the gap, so by asking for names that do not exist we can step from one real name to the next and list the whole zone. Plain DNS does not leak like this because it has no signed 'nothing exists here' record. The fix (NSEC3) proves non-existence over hashes of the names, so the walk only yields hashes. It costs the server a little extra work and only raises the bar — a determined attacker could still try to crack the hashes of a small zone.

Phase 6 — Attack on the last step (resolver to client)

Goal. DNSSEC is checked at the resolver, not on the user's machine. Move the attacker next to the client and forge the reply on the last short hop.

What we did. We moved the attacker to the client's network and forged the reply between the resolver and the client, and tested two kinds of client.

```
# Client A (an ordinary client that trusts its resolver):
$ dig example.com
;; flags: qr rd ra ad     <-- the attacker even turns on the 'ad' bit
example.com.  IN  A  172.16.1.30
$ curl http://example.com -> ** example.com has been HIJACKED **

# Client B (checks the signatures itself, holds the root key):
validating example.com/A: no valid signature found
RRSIG failed to verify
resolution failed        <-- the forgery is caught
```

Our observation. DNSSEC is checked at the resolver, but the short link from the resolver to the user carries no signature — only an 'ad' (authenticated) flag, which an attacker on that link can switch on freely. A normal client trusts that flag and the apparent sender, so it is fooled. A client that checks the signatures itself places its trust in the root key, not in the link, so it catches the forged answer. The lesson is that trust must sit where the checking actually happens.

Phase 7 — Encrypting the last step (DoH / DoT)

Goal. Encrypt and authenticate the link between the client and the resolver, then run the last-mile attack three ways against two kinds of client.

What we did. We set up a private certificate authority and gave the resolver a certificate for the name resolver.cs6008.local (a name only, no IP). The client installs our CA and reaches the resolver by that name over an encrypted link (DoH, over port 443). We also enabled DoT on port 853. We tested a strict client (encrypted only) and a permissive client (allowed to fall back to plain DNS).

```
# the resolver certificate is issued for the NAME only:
subject = CN=resolver.cs6008.local
SAN     = DNS:resolver.cs6008.local   (no IP address)

# verify: client resolves over the encrypted link, certificate valid (no -k):
result: real page      cleartext DNS on the wire: 0 packets (all encrypted)
upstream DNSSEC still holds: flags ... ad

# three attempts  x  two clients :
                          strict client       permissive client
(i)  change the answer    safe                safe
(ii) fake certificate     safe                safe
(iii) block the tunnel    no answer (safe)    FAKE PAGE (hijacked)
```

Our observation. Encrypting and authenticating the last link closes almost the whole gap. (i) The answer cannot be changed, because any change breaks the encryption. (ii) A fake server is caught, because the client checks the certificate against our own CA and the exact name. (iii) The only attempt that still works is against a client that is allowed to fall back to plain DNS: if the attacker blocks the encrypted link, that client quietly drops to plain DNS and can be fooled again, while a strict client simply stops. We chose DoH (port 443) because it blends in with normal web traffic and is harder to spot and block; DoT (port 853) is simpler but easy to block by its port. Even with all this, the resolver itself still sees every question — encryption hides the questions from the network, not from the resolver.

4. Summary of Results

Phase	Defence in place	Did the attack succeed?
1	None (baseline)	No attack - setup works and is isolated
2	None	Yes - client sent to the fake page
3	DNSSEC checked at the resolver	No - the fake answer is rejected
4	DNSSEC, set up correctly	No; but Yes if the DS link is missing
5	Signed 'does-not-exist' proof	Names leak; fixed by switching to NSEC3
6	DNSSEC up to the resolver only	Yes on a normal client; No if the client checks
7	Encrypted + checked last link	Only if the client allows a plain-DNS fallback

5. Conclusion

Each layer we added gave real protection, but no single layer was enough on its own:

• Plain DNS can be faked very easily by anyone on the path.
• DNSSEC stops fake answers, but only if the whole chain is set up fully, and only up to the resolver.
• The last step to the user needs its own protection — either the user's machine checks the signatures itself, or the link is encrypted and authenticated.
• Encryption stops both reading and changing on that link, but the client must not be allowed to quietly fall back to plain DNS.

One more point stands out across all the phases: even with every defence turned on, the resolver still sees every question the client asks. Hiding the questions from the resolver itself is a separate problem. Overall, good DNS security needs several layers working together, and every layer must be set up correctly — one missing record or one silent fallback is enough to let an attacker back in.

---

6. Repository and Work History

The complete source code for all seven phases, together with the commit history of the work, is available at:

https://github.com/ak23b0945/DNS_Lab